



Faculdade de Informática e Administração Paulista
Enterprise Challenge 2026 • Sociedade 5.0

SMART HAS — DIGITAL 360

Banco de Dados na Sociedade 5.0

Fase 6 — Oracle PL/SQL integrado ao back-end Java

Nome completo	RM
Eduardo Andrade Martins Vasques	556970
Otávio Ramos dos Santos Souza	550361
Enzo Miranda Ward de Paiva	557632
Rafael Pinto de Albuquerque	559136
Guilherme Leoni Vidigal Tibúrcio	557500

São Paulo • Outubro de 2026

Sumário

Resumo executivo

1. Parte 1 — Aprimoramento da solução
2. Parte 2 — Camada de persistência Oracle
3. Parte 3 — Functions e procedures PL/SQL
4. Integração Java (REST → JDBC → Oracle)
5. Validação e evidências
6. Como executar
7. Considerações finais e links

Resumo executivo

Esta entrega apresenta a Fase 6 do projeto **Smart HAS / Digital 360**, aplicativo de inclusão digital com a extensão **AI Logistics** (parceria Leroy Merlin). O foco desta fase foi dar ao sistema uma camada de dados profissional: um banco **Oracle** com 11 tabelas normalizadas, regras de negócio implementadas em **PL/SQL** (4 functions e 3 procedures) e uma **API Java (Spring Boot)** que aciona essas rotinas via **JDBC**.

A principal mudança arquitetural é que o motor de risco logístico, que antes existia apenas como heurística dentro do app, agora roda no banco de dados. Assim, o back-end e as rotinas automatizadas do banco passam a usar a mesma regra, e todo cálculo fica registrado para auditoria.

1. Parte 1 — Aprimoramento da solução

1.1 Evolução em relação à Fase 4

Antes (Fase 4)	Agora (Fase 6)	Valor agregado
Regra de risco só no código da aplicação (app, Python e Java)	Motor de risco no banco (FN_CALCULA_RISCO + SP_RECALCULAR_RISCO)	Regra única e reutilizável por qualquer cliente do banco
Sem persistência central	11 tabelas com PK, FK, CHECK e UNIQUE	Integridade garantida pelo banco, não pelo app
Sensores IoT apenas citados na topologia	Tabelas de sensores e leituras + geração automática de alertas	Monitoramento de carga sensível (temperatura, umidade, vibração)
Risco calculado e descartado	Histórico em T_SH_RISCO_HIST e alertas em T_SH_ALERTA	Rastreabilidade e auditoria de cada decisão
Sem visão por usuário	Relatório resumido por usuário (SP_GERAR_RELATORIO_USUARIOS)	Identifica quem precisa de apoio (tutoria) e quem tem pedidos em risco
Back-end Spring Boot da Fase 5 só com H2/JPA	Novo pacote p1sql no backend-java : JDBC para functions e procedures, com JWT, Swagger, tradução ORA → HTTP e testes	Mesmo back-end, agora com camada Oracle; o H2 da Fase 5 continua funcionando

Novas funcionalidades entregues nesta fase:

- **Alertas de telemetria IoT:** cada leitura fora da faixa segura do sensor gera um alerta ALTO ou CRÍTICO, sem duplicidade.
- **Recalcular risco com auditoria:** **POST /oracle/entregas/{id}/recalcular-risco** chama a procedure no Oracle, que grava o histórico e abre alerta quando o nível é ALTO ou CRÍTICO.
- **Relatório de engajamento e consumo por usuário:** trilhas, progresso médio, pedidos e alertas em aberto, com classificação ENGAJADO, EM_PROGRESSO ou INICIANTE.
- **Liga/desliga sem quebrar a Fase 5:** a camada Oracle só é ativada com **ORACLE_ENABLED=true**; sem ela, o **backend-java** sobe exatamente como antes, só com H2.

1.2 Refatoração e boas práticas aplicadas

- **Arquitetura em camadas na API:** Controller (REST) → Service (validação) → Repository (JDBC). Injeção de dependência por construtor.
- **Repository sem dependência de framework:** usa apenas `java.sql`, o que permite testá-lo com um DataSource simulado, sem banco no ar.
- **Erros padronizados:** as rotinas PL/SQL usam `RAISE_APPLICATION_ERROR` com códigos fixos (-20001 a -20003), que a API converte em HTTP 404 ou 400.
- **Transação controlada por quem chama:** as procedures não executam COMMIT; a API grava a leitura e dispara os alertas na mesma transação, com rollback em caso de falha.
- **PL/SQL robusto:** uso de `%TYPE/%ROWTYPE`, exceções nomeadas, `DETERMINISTIC`, cursores com `FOR UPDATE/WHERE CURRENT OF` e reaproveitamento de functions dentro das procedures.
- **Padrão de nomes:** prefixos T_SH_ (tabelas), FN_ (functions), SP_ (procedures), PK_/FK_/CK_/UK_ (constraints) e SEQ_SH_ (sequences).
- **Scripts reexecutáveis:** `99_drop_all.sql` remove tudo ignorando objetos inexistentes; `00_executar_tudo.sql` recria o ambiente em uma só execução.

2. Parte 2 — Camada de persistência Oracle

2.1 Modelo relacional (DER)

O modelo cobre os três domínios do Smart HAS: inclusão digital (usuários, trilhas e serviços), logística inteligente (pedidos, risco e alertas) e IoT (sensores e leituras). As relações N:N (usuário × curso e usuário × serviço) foram resolvidas com as tabelas associativas `T_SH_MATRICULA` e `T_SH_FAVORITO`.

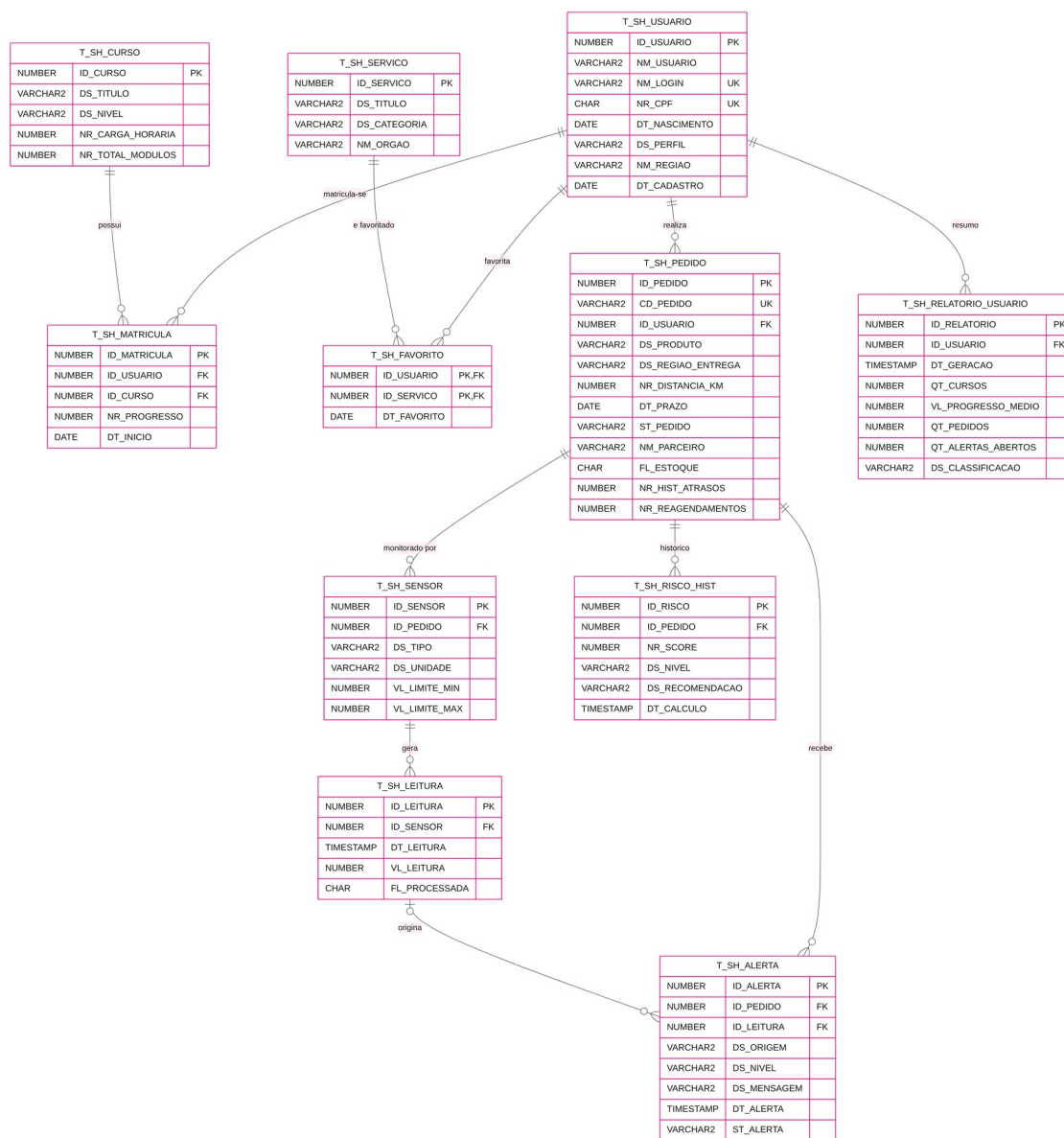


Figura 1 — DER do Smart HAS / Digital 360 (Fase 6). Arquivo em alta resolução: database-oracle/DER_SmarHAS_Fase6.png

2.2 Tabelas criadas

Tabela	Finalidade	Chaves e restrições principais
T_SH_USUARIO	Usuários do app (idosos, adultos, tutores)	PK; UK em login e CPF; CHECK no perfil
T_SH_CURSO	Trilhas de aprendizagem	PK; CHECK no nível e em carga/módulos > 0
T_SH_MATRICULA	Usuário × curso com progresso	FK usuário e curso; UK (usuário, curso); progresso 0–100
T_SH_SERVICO	Guia de serviços públicos	PK
T_SH_FAVORITO	Serviços favoritos do usuário	PK composta (usuário, serviço); 2 FKs
T_SH_PEDIDO	Pedidos da AI Logistics	UK no código; FK usuário; CHECK em status, estoque e números ≥ 0

Tabela	Finalidade	Chaves e restrições principais
T_SH_SENSOR	Sensores IoT na carga	FK pedido; CHECK no tipo e limite máx > mín
T_SH_LEITURA	Telemetria dos sensores	FK sensor; flag de processamento S/N
T_SH_ALERTA	Alertas de sensor ou de risco	FK pedido e leitura (opcional); CHECK em origem, nível e status
T_SH_RISCO_HIST	Histórico do motor de risco	FK pedido; score 0–100; CHECK no nível
T_SH_RELATORIO_USUARIO	Snapshot do relatório por usuário	FK usuário; CHECK na classificação

Também foram criados 6 índices de apoio às chaves estrangeiras e às consultas de leituras pendentes, além de comentários de dicionário (**COMMENT ON TABLE**) e 6 sequences para os IDs gerados pela aplicação.

2.3 Dados simulados importados

O script **02_dados_simulados.sql** carrega dados fictícios coerentes com o app. Os pedidos 1 a 4 são os mesmos exibidos pelo app Flutter, para que a demonstração mostre os mesmos dados no app e no banco.

Tabela	Registros	Destaque
Usuários	6	Nomes e CPFs fictícios; perfis IDOSO, ADULTO e TUTOR
Cursos / Matrículas	4 / 9	Progresso de 0% a 100%
Serviços / Favoritos	4 / 5	INSS, SUS, gov.br e Caixa
Pedidos	5	Um pedido ATRASADO e sem estoque (cenário crítico)
Sensores	5	Temperatura (tinta), umidade (pisso, ferramentas) e vibração (furadeira, ventilador)
Leituras	19	6 leituras fora da faixa segura, para disparar os alertas

3. Parte 3 — Functions e procedures PL/SQL

3.1 Functions

Function	Tipo	Parâmetros / retorno	Exceções tratadas
FN_CALCULA_RISCO	Indicador	IN p_id_pedido → NUMBER (score 0–100)	-20001 pedido inexistente; -20002 parâmetro nulo
FN_PROGRESSO_MEDIO_USUARIO	Indicador	IN p_id_usuario → NUMBER (média de progresso)	-20003 usuário inexistente; -20002 nulo
FN_NIVEL_RISCO	Classificação (DETERMINISTIC)	IN p_score → VARCHAR2 (BAIXO a CRITICO)	-20002 score fora de 0–100

Function	Tipo	Parâmetros / retorno	Exceções tratadas
FN_RESUMO_PEDIDO	Dados formatados	IN p_id_pedido → VARCHAR2 (linha pronta para exibição)	-20001; -20002

Regra do indicador de risco (FN_CALCULA_RISCO): +18 por atraso no histórico, +12 por reagendamento, +25 sem estoque, +15 se a distância for maior que 20 km (+8 se maior que 10 km) e +20 se o pedido já estiver ATRASADO, com limite de 100. É a mesma regra que o app usava, agora centralizada no banco.

```
CREATE OR REPLACE FUNCTION FN_CALCULA_RISCO (p_id_pedido IN T_SH_PEDIDO.ID_PEDIDO%TYPE)
RETURN NUMBER IS
    v_pedido T_SH_PEDIDO%ROWTYPE; v_score NUMBER := 0; e_param_nulo EXCEPTION;
BEGIN
    IF p_id_pedido IS NULL THEN RAISE e_param_nulo; END IF;
    SELECT * INTO v_pedido FROM T_SH_PEDIDO WHERE ID_PEDIDO = p_id_pedido;
    v_score := v_pedido.NR_HIST_ATRASOS * 18 + v_pedido.NR_REAGENDAMENTOS * 12;
    IF v_pedido.FL_ESTOQUE = 'N' THEN v_score := v_score + 25; END IF;
    ... -- distância e status ATRASADO (ver 03_functions.sql)
    RETURN LEAST(v_score, 100);
EXCEPTION
    WHEN e_param_nulo THEN RAISE_APPLICATION_ERROR(-20002, '...parametro nulo');
    WHEN NO_DATA_FOUND THEN RAISE_APPLICATION_ERROR(-20001, '...pedido nao encontrado');
END;
```

Uso prático em consultas SQL (script [05_testes_e_consultas.sql](#)). As functions aparecem em SELECT, WHERE, ORDER BY, em subconsultas com agregação e na view **V_SH_PEDIDO_RISCO**, que é a consulta usada pela API.

```
-- Pedidos que exigem ação: function usada no WHERE
SELECT FN_RESUMO_PEDIDO(ID_PEDIDO) AS RESUMO
FROM T_SH_PEDIDO
WHERE FN_NIVEL_RISCO(FN_CALCULA_RISCO(ID_PEDIDO)) IN ('ALTO', 'CRITICO');
```

Pedido	Status	Score esperado	Nível
LM-2026-0002 (tinta 18L)	ATRASADO, sem estoque	100	CRITICO
LM-2026-0005 (ventilador)	EM_TRANSITO, 25 km	45	MEDIO
LM-2026-0003 (furadeira)	PENDENTE, 14 km	26	MEDIO
LM-2026-0004 (piso)	ENTREGUE, 19 km	8	BAIXO
LM-2026-0001 (ferramentas)	EM_TRANSITO, 8 km	0	BAIXO

Resultado esperado de **FN_RESUMO_PEDIDO(2)**: **LM-2026-0002 | Tinta acrílica 18L | ATRASADO | Risco 100 (CRITICO) | Prazo 15/06/2026 | Parceiro Total Express.**

3.2 Procedures

SP_REGISTRAR_ALERTAS_SENSORES (p_id_pedido IN, p_qt_alertas OUT) — rotina automatizada de monitoramento IoT. Um cursor **FOR UPDATE** percorre as leituras ainda não processadas; para cada leitura fora da faixa do sensor, um IF/ELSIF classifica o desvio (até 20% da amplitude: ALTO; acima disso: CRÍTICO) e insere o alerta. A leitura é marcada como processada com **WHERE CURRENT OF**, o

que torna a rotina idempotente: rodar de novo não duplica alertas. Resultado esperado com os dados simulados: **6 alertas**.

SP_RECALCULAR_RISCO (p_id_pedido IN; p_score, p_nivel, p_recomendacao, p_mensagem_cliente OUT) — o motor de risco da AI Logistics. Reaproveita **FN_CALCULA_RISCO** e **FN_NIVEL_RISCO**, define a recomendação, grava o histórico e, quando o nível é ALTO ou CRÍTICO, abre um alerta de risco apenas se não houver outro aberto para o mesmo pedido (gestão por exceção). Os parâmetros OUT seguem o contrato **RiscoLogistico** que o app já consumia. É a **procedure acionada pelo back-end Java**.

SP_GERAR_RELATORIO_USUARIOS (p_id_usuario IN, p_qt_linhas OUT) — relatório resumido de consumo e engajamento por usuário. Usa um cursor explícito com parâmetro (OPEN/FETCH/EXIT WHEN %NOTFOUND/CLOSE), calcula as métricas, classifica o usuário, grava um snapshot em **T_SH_RELATORIO_USUARIO** e imprime o resumo com DBMS_OUTPUT. No tratamento de exceção, fecha o cursor se ele ainda estiver aberto (%ISOPEN).

Usuário	Trilhas	Progresso médio	Pedidos	Alertas abertos	Classificação
Maria Aparecida Lima	3	35	1	3	EM_PROGRESSO
José Carlos Pereira	2	55	1	0	EM_PROGRESSO
Ana Beatriz Souza	2	75	1	1	ENGAJADO
Antônio Ferreira	1	20	1	2	INICIANTE
Lúcia Helena Costa	1	45	1	1	EM_PROGRESSO
Pedro Henrique Alves	0	0	0	0	INICIANTE

Tabela 1 — Resultado esperado do relatório após a execução das procedures de alerta e de risco (script 05).

3.3 Conceitos de PL/SQL utilizados

Conceito	Onde aparece
Parâmetros IN / OUT e RETURN	Todas as functions (IN + RETURN); procedures com IN e OUT
IF / ELSIF / ELSE	FN_NIVEL_RISCO, FN_CALCULA_RISCO, classificação dos alertas e do relatório
CURSOR FOR LOOP com FOR UPDATE	SP_REGISTRAR_ALERTAS_SENSORES
Cursor explícito com parâmetro + LOOP / EXIT WHEN	SP_GERAR_RELATORIO_USUARIOS
EXCEPTION (nomeadas, NO_DATA_FOUND, OTHERS)	Todas as rotinas, com RAISE_APPLICATION_ERROR padronizado
%TYPE / %ROWTYPE	Parâmetros e variáveis ligados às colunas das tabelas
Função local e composição de functions	fmt() na procedure de alertas; FN_RESUMO_PEDIDO usa FN_CALCULA_RISCO

4. Integração Java (REST → JDBC → Oracle)

A integração foi feita **dentro do back-end Spring Boot da Fase 5** (`backend-java/`), no novo pacote `com.smarthas.backend.plsql`. Os endpoints ficam sob `/oracle/**`, protegidos pelo mesmo login JWT e documentados no Swagger, e chamam as rotinas PL/SQL por meio de `CallableStatement`. O pool de conexões Oracle é próprio do pacote, então o H2 e o JPA da Fase 5 continuam intactos.

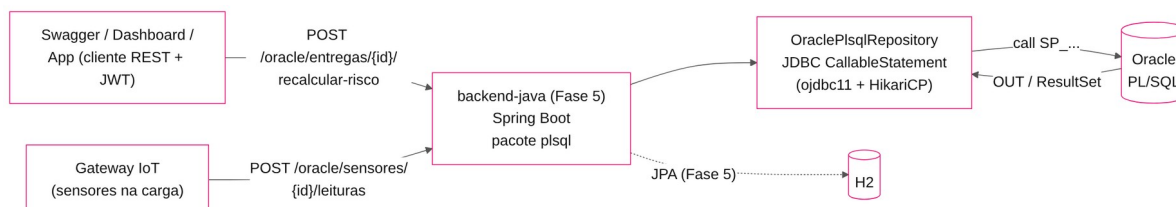


Figura 2 — Fluxo de integração: requisição REST → backend-java (pacote plsql) → JDBC → procedure Oracle.

Endpoint	Rotina Oracle acionada
GET /oracle/pedidos	View V_SH_PEDIDO_RISCO (FN_CALCULA_RISCO, FN_NIVEL_RISCO, FN_RESUMO_PEDIDO)
POST /oracle/entregas/{id}/recalcular-risco	SP_RECALCULAR_RISCO (evento de back-end)
POST /oracle/sensores/{id}/leituras	INSERT da leitura + SP_REGISTRAR_ALERTAS_SENDORES na mesma transação
GET /oracle/alertas?status=ABERTO	Consulta T_SH_ALERTA
POST /oracle/relatorios/usuarios	SP_GERAR_RELATORIO_USUARIOS

```
try (Connection con = dataSource.getConnection();
    CallableStatement cs =
        con.prepareCall("{call SP_RECALCULAR_RISCO(?, ?, ?, ?, ?)}")) {
    cs.setLong(1, idPedido);
    cs.registerOutParameter(2, Types.NUMERIC); // score
    cs.registerOutParameter(3, Types.VARCHAR); // nivel
    cs.registerOutParameter(4, Types.VARCHAR); // recomendacao
    cs.registerOutParameter(5, Types.VARCHAR); // mensagem ao cliente
    cs.execute();
    return new RiscoLogistico(idPedido, cs.getInt(2),
        cs.getString(3), cs.getString(4), cs.getString(5));
} catch (SQLException e) {
    // ORA-20001 -> HTTP 404 | ORA-20002 -> HTTP 400
    throw OracleErrorTranslator.traduzir("recalcular risco", e);
}
```

Exemplo de resposta de `POST /oracle/entregas/2/recalcular-risco`: `{"pedidoId":2, "riscoScore":100, "riscoNivel":"CRITICO", "recomendacao":"Acionar suporte logistico...", "mensagemCliente":"Detectamos fatores..."}`.

5. Validação e evidências

Toda a solução foi executada de ponta a ponta em **03/10/2026**, com **Oracle Database 23ai Free** em Docker (imagem `gvenzl/oracle-free`, SQL*Plus 23.26) e o `backend-java` em container

Maven/Temurin 17 na mesma rede Docker. Os registros completos estão na pasta **evidencias/** da entrega.

Verificação	Resultado obtido
Criação do schema (00_executar_tudo.sql)	11 tabelas, 6 sequences, 6 índices, 4 functions, 3 procedures e 1 view; todos VALID em USER_OBJECTS e USER_ERRORS vazio
Carga de dados simulados	USUARIO 6 • CURSO 4 • MATRICULA 9 • SERVICO 4 • FAVORITO 5 • PEDIDO 5 • SENSOR 5 • LEITURA 19
Functions em consultas SQL	Scores 100 / 45 / 26 / 8 / 0 e resumo formatado idênticos aos esperados
SP_REGISTRAR_ALERTAS_SENSORES	6 alertas gerados; reexecução gerou 0 (idempotente)
SP_RECALCULAR_RISCO	5 cálculos no histórico; 1 alerta de risco (pedido 0002) sem duplicar no recálculo
SP_GERAR_RELATORIO_USUARIOS	Relatório idêntico à Tabela 1 (DBMS_OUTPUT e snapshot)
Tratamento de exceções	ORA-20001, ORA-20002, ORA-20003 e ORA-20002 retornados nos 4 testes
backend-java: mvn test (pacote plsql)	11 testes, 0 falhas, BUILD SUCCESS
backend-java com ORACLE_ENABLED=true	Login JWT ok • /oracle/pedidos 200 • recalcular-risco 200 (CRITICO, 100) • leitura de 60 °C 201 com 1 alerta • /oracle/alertas 200 • /oracle/relatorios/usuarios 200 • /pedidos da Fase 5 (H2) continua 200
Erros e segurança via HTTP	Pedido 999 → 404 • status inválido → 400 • sensor 99 → 404 • sem token → 403

O script **database-oracle/testar_no_docker.ps1** repete toda essa verificação com um único comando (requer Docker Desktop).

6. Como executar

Banco: em qualquer Oracle 11g ou superior (servidor da faculdade, Oracle Free em Docker ou Oracle Live SQL para a parte PL/SQL), execute **database-oracle/00_executar_tudo.sql** ou rode os scripts 99, 01, 02, 03, 04 e 05 nessa ordem (F5 no SQL Developer). Oracle Free em Docker:

```
docker run -d --name oracle-free -p 1521:1521 -e ORACLE_PASSWORD=Fiap2026 \
-e APP_USER=smarthas -e APP_USER_PASSWORD=smarthas gvenzl/oracle-free:slim-faststart
# conexao: smarthas/smarthas @ localhost:1521/FREEPDB1
```

Back-end: em **backend-java/**, defina **ORACLE_ENABLED=true**, **ORACLE_URL**, **ORACLE_USER** e **ORACLE_PASSWORD** e rode **mvn spring-boot:run**. Faça login em **POST /auth/usuarios/login** (admin/admin123) e use o token nas rotas **/oracle/****, ou teste tudo pelo Swagger (**/swagger-ui.html**). Os testes rodam com **mvn test** sem precisar de banco.

Tudo de uma vez: **database-oracle/testar_no_docker.ps1** sobe o Oracle, executa os scripts, roda os testes, compila e sobe o **backend-java** com a camada Oracle ligada e chama todos os endpoints.

7. Considerações finais e links

A Fase 6 tornou o Smart HAS mais robusto ao levar para o banco as regras que exigem consistência e auditoria (risco, alertas e relatórios) e ao conectar essas regras ao back-end Java com tratamento de erros e controle de transação. Próximos passos: uma tela no dashboard Angular consumindo `/oracle/alertas` e `/oracle/relatorios/usuarios`, o app consumindo as rotas `/oracle/**` direto e um job agendado (DBMS_SCHEDULER) para executar a rotina de alertas periodicamente.

Repositório GitHub: https://github.com/DevRafonalde/digital360_flutter (scripts em `database-oracle/`, integração em `backend-java/`, documentos em `docs/fase6/`)

Vídeo de apresentação (YouTube, não listado): [INSERIR LINK]